

CAWSR: Carla-AutoWare Scenario Runner

David Gasinski¹, Olek Osikowicz¹, Gwilym Rutherford¹, and
Donghwan Shin¹

¹ The University of Sheffield

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#))

Summary

CAWSR (CARLA-AutoWare-Scenario Runner) facilitates the simulation-based testing of the open-source autonomous driving system, Autoware, within CARLA, the state-of-the-art open-source driving simulator. Building on existing tools, this project introduces a research-oriented testing framework for the execution of complex driving scenarios, as well as supporting the implementation of a wide range of verification strategies.

Statement of Need

Verifying Autonomous Driving Systems (ADS) is a critical step before they can be deployed. However, relying only on real-world testing is too expensive, inefficient, and potentially dangerous. Consequently, simulation-based testing has become essential, allowing researchers to safely test driving agents against critical situations at scale. Among these tools, CARLA ([Dosovitskiy et al., 2017](#)) has become the de-facto standard in the research community due to its rich ecosystem of open-source tools, benchmarks, and documentation.

Currently, the standard for evaluating ADS in CARLA is the CARLA Leaderboard and its engine, Scenario Runner (SR) ([CARLA, 2025](#)). This framework is typically used to test “black-box” driving agents, such as ML-based systems which expose only sensor-level inputs and driving control outputs. By running a set of predefined, challenging driving scenarios, researchers can systematically assess agent performance using common metrics like driving score, infractions, and route completion. However, applying this testing framework to industry-grade ADS, such as Autoware ([Kato et al., 2018](#)) or Apollo ([Baidu, 2017](#)), remains difficult. Although communication bridges exist between CARLA and these systems ([Guardstrikelab, 2023](#); [Kaljavesi et al., 2024](#)), they lack native support for scenario execution engines, which limits their utility for scenario-based testing.

This gap has created a significant bottleneck for the research community. Previously, researchers developing scenario generation algorithms mainly relied on combining Apollo with the LGSVL simulator ([Rong et al., 2020](#)). However, LGSVL is now outdated, with official support ending in January 2022. This leaves many researchers without a suitable industry-grade “subject” for evaluating their algorithms.

CAWSR aims to bridge this gap by enabling the evaluation of Autoware in complex driving scenarios within CARLA. By building on the established CARLA platform, this work provides a modern replacement for the outdated Apollo/LGSVL workflow. It also allows Autoware to be directly compared with state-of-the-art research agents on the CARLA Leaderboard.

Effective ADS verification requires the ability to systematically explore the operational design domain. To support this, CAWSR provides a flexible interface for algorithmic scenario generation. This facilitates a wide range of verification strategies based on common metrics, such as the CARLA Leaderboard's driving score ([CARLA Team, 2024](#)).

41 Lastly, it is worth noting that simulators can often introduce unintended nondeterminism,
42 which leads to inconsistent test results ([Chance et al., 2022](#); [Osikowicz et al., 2025](#)). Therefore,
43 CAWSR is designed to minimise such nondeterminism throughout the evaluation pipeline.

44 State of the Field

45 Simulation-based testing of Autonomous Vehicles spans several tools and frameworks, each
46 targeting different aspects of the evaluation pipeline. CAWSR occupies a unique niche at the
47 intersection of industry-grade ADS integration, scenario-based testing, and modern simulator
48 support.

49 Simulation Platforms

50 CARLA ([Dosovitskiy et al., 2017](#)) has established itself as the standard open-source simulator
51 for ADS research, offering a rich ecosystem of tools, high-fidelity sensor models, and an active
52 community. Its closest historical competitor for industry-grade ADS testing was LGSVL ([Rong
53 et al., 2020](#)), a Unity-based simulator developed by LG Electronics that provided integration
54 with Apollo. However, LGSVL's official support was discontinued in January 2022, leaving
55 a significant gap for researchers who relied on it as the simulation backend for their testing
56 pipelines.

57 Scenario Execution Engines

58 Within the CARLA ecosystem, Scenario Runner (SR) ([CARLA, 2025](#)) is the standard scenario
59 execution engine, underpinning the CARLA Leaderboard([CARLA Team, 2024](#)) evaluation
60 framework. SR is well-suited to evaluating black-box ML-based agents that expose only
61 sensor inputs and control outputs. However, it is not designed to interface with full-stack,
62 industry-grade ADS such as Autoware, which rely on complex middleware architectures (e.g.,
63 ROS2) for internal communication. CAWSR extends the SR execution model specifically
64 to accommodate Autoware's architecture, enabling scenario-based testing and evaluation of
65 module-based ADS, which SR does not support.

66 Autoware Integration Tools

67 Three existing tools provide partial integration between Autoware and CARLA. The CARLA-
68 Autoware Bridge ([Kaljavesi et al., 2024](#)) handles low-level data translation between the simulator
69 and Autoware's ROS2 interface, converting CARLA snapshots and sensor data into Autoware's
70 coordinate system. PCLA ([Tehrani et al., 2025](#)) takes a different approach, simplifying the
71 deployment of multiple pretrained CARLA Leaderboard agents across CARLA versions, making
72 it a valuable benchmarking utility for comparing ML agents. However, PCLA abstracts away
73 ADS implementation details and does not provide the deep simulator-agent integration needed
74 for route-based scenario execution, alongside limited support for module-based driving systems.

75 Most recently, autoware_carla_leaderboard ([Kaljavesi et al., 2026](#)), has been developed by the
76 same group behind the CARLA-Autoware Bridge, introducing support for executing CARLA
77 Leaderboard scenarios with Autoware Universe. This represents a concurrent development
78 with overlapping goals. However, it focuses on execution of predefined scenario sets and does
79 not provide a programmatic interface for algorithmic scenario generation, which is a primary
80 design goal of CAWSR. Furthermore, as this tool was released after the initial development of
81 CAWSR, the two works are best understood as complementary efforts addressing the same
82 research gap independently. Neither tool exposes a programmatic interface for algorithmic
83 scenario generation, a core requirement for systematic verification research.

84 Build vs. Contribute Justification

85 The existing landscape presents a clear rationale for CAWSR as a new tool rather than
86 a contribution to an existing project. Extending SR to support Autoware would require
87 fundamental architectural changes to its agent model, which is designed around the black-box
88 paradigm. The CARLA-Autoware Bridge provides a necessary but insufficient building block
89 that CAWSR builds on top of as a dependency. No existing tool unifies a maintained, modern
90 simulator with an industry-grade ADS subject, supporting route-based scenario execution and
91 a flexible interface for programmatic scenario generation. CAWSR is the first framework to
92 integrate all four, enabling researchers to systematically evaluate Autoware against state-of-
93 the-art scenario generation algorithms on a reproducible testing platform.

94 Research Impact

95 CAWSR provides a significant research impact by bridging the gap between the widely-used
96 CARLA simulator and the industry-grade Autoware ADS. It addresses a critical bottleneck
97 in the ADS testing research community by offering a modern replacement for the outdated
98 Apollo/LGSVL workflow, which has lacked official support since 2022.

99 It enhances research reproducibility by implementing a fully synchronous evaluation pipeline
100 that minimises unintended nondeterminism in simulation-based testing. To ensure community
101 readiness and ease of use, it is distributed as a Docker container.

102 Furthermore, by adopting CARLA Leaderboard metrics, CAWSR enables researchers to directly
103 compare Autoware with other state-of-the-art driving agents in the CARLA environment. It
104 provides an essential foundation for the systematic evaluation of various testing approaches for
105 ADS.

106 Software Design

107 CAWSR is a fully synchronous testing framework that directly integrates the CARLA simulator,
108 Scenario Runner (as the scenario executor), and Autoware (as the System Under Test) to
109 facilitate autonomous driving testing research. The tool is distributed as a containerized
110 deployment using Docker to manage complex dependencies and simplify the setup process.
111 Currently, two modes of operation are supported:

- 112 1. *Scenario Generation Mode*: Enables the dynamic generation and execution of scenarios
113 (e.g. iterative scenario generation) provided by a user-defined algorithm. This is
114 particularly useful for assessing the performance of new simulation-based ADS testing
115 techniques.
- 116 2. *Benchmark Mode*: Allows the execution of a predefined set of scenario definitions provided
117 by the user. This is useful for standardised evaluations and comparisons between different
118 driving agents.

119 The evaluation pipeline is engineered to be fully synchronous, minimising unintentional non-
120 determinism to facilitate reproducible results. However, it is noted that minor variations may
121 still persist due to inherent non-determinism in upstream dependencies, such as the driving
122 simulator or the driving agent itself (Chance et al., 2022; Osikowicz et al., 2025).

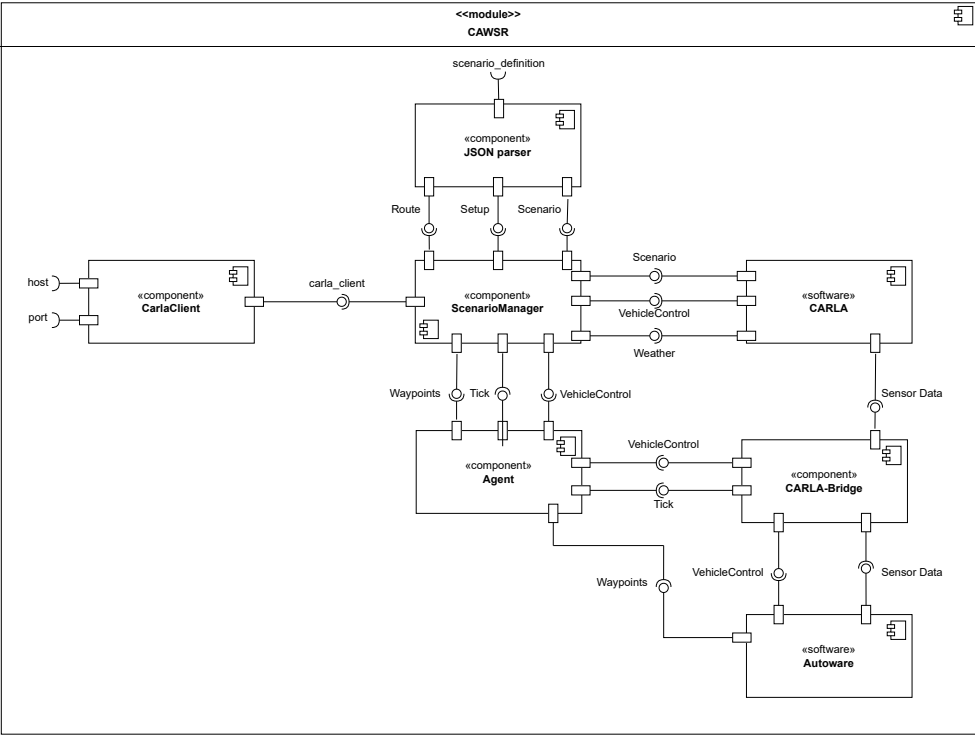


Figure 1: Internal component diagram of CAWSR.

- Figure 1 illustrates the CAWSR architecture and its fundamental components. The framework operates through four primary modules:
- **CarlaClient**: A native CARLA PythonAPI class that establishes a TCP connection (via host IP and port). It serves as the framework's exclusive interface for extracting simulation data and spawning entities.
 - **JSON Parser**: Translates the `scenario_definition` (see Figure 2) into a Behavior Tree (BT). It utilises Scenario Runner's *Atomic Behaviours* and *Atomic Conditions* as modular primitives to define discrete actions (e.g., spawning pedestrians) and logic triggers.
 - **ScenarioManager** (CARLA, 2025): Orchestrates the simulation loop by evaluating the BT to update actor states and triggering CARLA simulation ticks. Execution terminates based on CARLA Leaderboard criteria (CARLA Team, 2024), as summarised in Table 1. Post-execution, the module calculates the Driving Score (DS) according to the official leaderboard metrics.
 - **Agent and CarlaBridge**: The Agent manages the ROS2 connection to Autoware. At each timestep, the CarlaBridge (Kaljavesi et al., 2024) transforms CARLA snapshots and sensor data into the Autoware coordinate system. Autoware processes these inputs to issue control commands, which the Agent then applies to the ego vehicle.

Table 1: Termination Criteria of each scenario within CAWSR.

Termination Criteria	Description
Route_Completion	Agent reached the end of the route.
Actor_Blocked	Agent is blocked, not moving for 180s.
Simulation_Timeout	No client-server communication established (30s).

To facilitate development, we introduce a new domain model for the definition of route-based scenarios within CARLA, described in Figure 2, alongside a JSON implementation. This model is based on the format introduced by Scenario Runner, facilitating support between both frameworks.

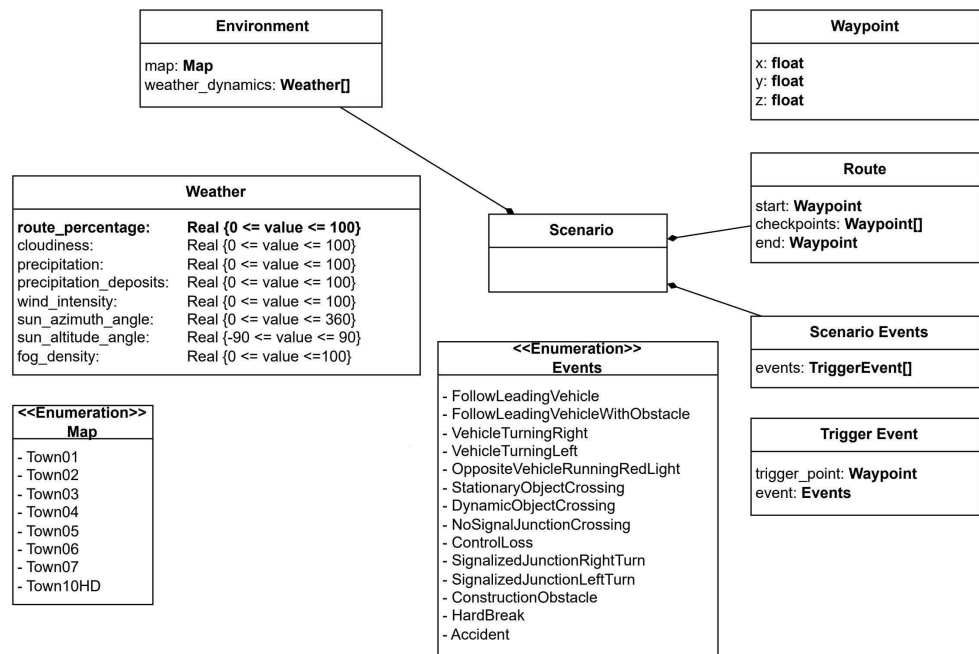


Figure 2: Scenario definition domain model.

Conclusion

To summarise, CAWSR provides ADS testing research community an easy to use Autoware evaluation pipeline. We hope that this work can facilitate the evaluation of new testing approaches on a state of the art driving system.

AI Usage Disclosure

Generative AI tools were used in this work solely to support high-level research concepts and structural ideas. All software implementation, including the source code, architecture, and deployment scripts, was authored entirely by the researchers without AI assistance.

Acknowledgements

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No. RS-2025-02218761, 50%) and by the Engineering and Physical Sciences Research Council (EPSRC) [EP/Y014219/1].

References

Baidu. (2017). *Apollo: Open Source Autonomous Driving*. <https://github.com/ApolloAuto/apollo>.

- 159 CARLA. (2025). *Scenario Runner: Traffic Scenario Definition and Execution Engine for CARLA*.
160 GitHub; https://github.com/carla-simulator/scenario_runner. https://github.com/carla-simulator/scenario_runner
161
- 162 CARLA Team. (2024). *Get started with leaderboard 2.0. CARLA autonomous driving*
163 *leaderboard*. https://leaderboard.carla.org/get_started_v2_0/
- 164 Chance, G., Ghobrial, A., McAreavey, K., Lemaignan, S., Pipe, T., & Eder, K. (2022). On
165 determinism of game engines used for simulation-based autonomous vehicle verification.
166 *IEEE Transactions on Intelligent Transportation Systems*, 23(11), 20538–20552. <https://doi.org/10.1109/TITS.2022.3177887>
167
- 168 Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., & Koltun, V. (2017). CARLA: An open
169 urban driving simulator. *Proceedings of the 1st Annual Conference on Robot Learning*,
170 1–16.
- 171 Guardstrikelab. (2023). *carla_apollo_bridge: Data and Control Bridge for Apollo and Carla*.
172 GitHub; https://github.com/guardstrikelab/carla_apollo_bridge.
- 173 Kaljavesi, G., Kerbl, T., Betz, T., Mitkovskii, K., & Diermeyer, F. (2024). *CARLA-autoware-*
174 *bridge: Facilitating autonomous driving research with a unified framework for simulation*
175 *and module development*. 224–229. <https://doi.org/10.1109/iv55156.2024.10588623>
- 176 Kaljavesi, G., Majstorovic, D., Clement, M., & Diermeyer, F. (2026). *Empirical mapping of*
177 *technical bottlenecks in autonomous driving: A cross-platform evaluation of simulated and*
178 *real-world performance*.
- 179 Kato, S., Tokunaga, S., Maruyama, Y., Maeda, S., Hirabayashi, M., Kitsukawa, Y., Monrroy, A.,
180 Ando, T., Fujii, Y., & Azumi, T. (2018). Autoware on board: Enabling autonomous vehicles
181 with embedded systems. *2018 ACM/IEEE 9th International Conference on Cyber-Physical*
182 *Systems (ICCPs)*, 287–296. <https://doi.org/10.1109/iccps.2018.00035>
- 183 Osikowicz, O., McMinn, P., & Shin, D. (2025). Empirically evaluating flaky tests for
184 autonomous driving systems in simulated environments. *2025 IEEE/ACM International*
185 *Flaky Tests Workshop (FTW)*, 13–20. <https://doi.org/10.1109/FTW66604.2025.00009>
- 186 Rong, G., Shin, B. H., Tabatabaee, H., Lu, Q., Lemke, S., Možeiko, M., Boise, E., Uhm,
187 G., Gerow, M., Mehta, S., Agafonov, E., Kim, T. H., Sterner, E., Ushiroda, K., Reyes,
188 M., Zelenkovsky, D., & Kim, S. (2020). LGSVL simulator: A high fidelity simulator for
189 autonomous driving. *2020 IEEE 23rd International Conference on Intelligent Transportation*
190 *Systems (ITSC)*, 1–6. <https://doi.org/10.1109/ITSC45102.2020.9294422>
- 191 Tehrani, M. J., Kim, J., & Tonella, P. (2025). PCLA: A framework for testing autonomous
192 agents in the CARLA simulator. *Proceedings of the 33rd ACM International Conference on*
193 *the Foundations of Software Engineering*, 1040–1044. <https://doi.org/10.1145/3696630.3728577>
194